



university of
groningen

Quantitative text analysis – Essex Summer School

Weak supervision, transfer learning, and zero-shot classification

dr. Martijn Schoonvelde

University of Groningen

Today's class

- Question: how can we classify text when high-quality labelled data are scarce?
- Three answers in today's readings:
 - **weak supervision**: cheaper but noisier labels
 - **transfer learning**: reusing language and task knowledge
 - Example: **NLI-based zero-shot classification**
- Lab session: first learn the workflow of a transfer learning process, then (optionally) run the transformer model

The label bottleneck

Traditional supervised text analysis starts with labelled examples for the task at hand.

- High-quality labels are expensive
 - domain expertise
 - codebook development
 - intercoder reliability
 - validation and adjudication
- Many research questions need more labels than we can realistically code by hand
- New approaches (try to) reduce the cost of supervision without abandoning validation

Two routes to cheaper supervision

	Weak supervision	Transfer learning
Core idea	generate labels using heuristics	reuse a pre-trained model
Data logic	many noisy labels	fewer high-quality labels
Cost reduction	cheaper labelling at scale	training-data requirements
Pipeline stage	label data first, then train	start with a trained model, then adapt
Main risk	noisy labels contain bias	transferred knowledge may not fit the task

Both reduce reliance on large gold-standard training sets, but neither **removes the need for validation**.

Weak supervision

- Weak supervision replaces manual labelling one case at a time with cheaper, scalable signals
- Typical sources:
 - keyword dictionaries
 - rule-based patterns or metadata
 - crowd labels or expert shortcuts
 - labels from another model
- The aim is to combine noisy signals into something useful for:
 - triage and validation
 - training a downstream classifier

Label functions

A label function is a rule-based or model-based procedure that casts a **weak vote**, abstains, or returns a score. It can contain one decision rule or a set of decision rules.

Label function	Possible decision rules	Output
Economy keywords	tax, budget, inflation, wages, growth	economy / abstain
Cost-of-living phrases	energy bills, food prices, living costs	economy / abstain
Fiscal-policy rule	mentions spending and Treasury in same speech	economy / abstain
Model-based LF	existing classifier predicts economy with high confidence	economy / abstain

The terms inside a label function are the evidence that makes it fire. The label function itself is the procedure that outputs a weak signal.

Reading label-function output

Each label function looks at the same speech and returns a weak label, an abstention, or sometimes a confidence score.

Speech	Economy LF	Health LF	Procedural LF	What do we learn?
Budget statement	economy	abstain	abstain	likely economy
Mixed debate	economy	health	abstain	inspect ambiguity
No keyword match	abstain	abstain	abstain	coverage gap

- Agreement can increase confidence
- Disagreement identifies cases for human inspection
- Abstention reveals what weak supervision cannot see

Weak supervision pipeline

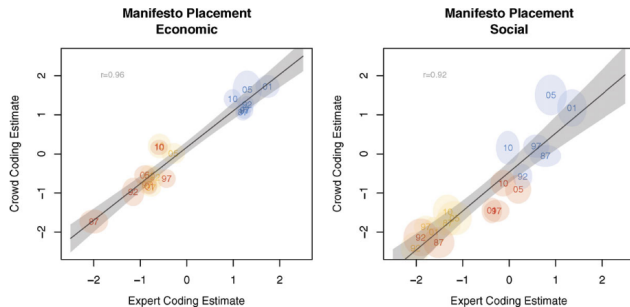
Step	What happens
1. Write label functions	rules, dictionaries, models, metadata signals
2. Create a label matrix	rows = documents; columns = label-function outputs
3. Combine noisy votes	resolve agreement, conflict, and abstention
4. Produce probabilistic labels	use weak labels for training
5. Validate	estimate errors against human-coded cases

Bottom line: weak supervision builds a cheaper labelled dataset. It does not make the labels automatically correct.

Crowdsourcing as weak supervision

- Benoit *et al.* (2016) show that non-experts can produce useful political text labels
- Individual crowd labels are noisy but aggregated judgements can be reliable and scalable

FIGURE 3. Expert and Crowd-sourced Estimates of Economic and Social Policy Positions



Why classical classifiers need many labels

- Classical supervised classifiers usually start from a document-feature matrix
- They learn from task-specific examples:
 - which words predict which class?
 - how should word counts be weighted?
- This can work well when we have enough labelled data but **it struggles when concepts are rare, classes are imbalanced, or the corpus changes between the training and the test set**

Based on the representation problem discussed in Wankmueller (2022) and the domain-generalisation problem discussed in Kroon *et al.* (2024).

Drawbacks of traditional supervised learning

The key disadvantage of classical machine learning algorithms is that they start the training process **without any prior 'knowledge' of language or tasks** (Laurer *et al.* 2024)

- Humans know that the words attack and invasion express similar meanings, or that the words happy and not happ tend to appear in different contexts

Conventional models on the other hand need **to learn these language patterns and tasks from scratch** with the training data as the only source of information.

From BoW features to learned representations

Approach	What the model starts with	Main limitation
Dictionary BoW classifier	researcher-defined words word counts from labelled examples	brittle outside target domain little prior language knowledge
Word embeddings	word vectors (possibly) learned elsewhere	one vector per word type
Transformer	contextual representations learned elsewhere	powerful but harder to inspect

From static to contextual representations

- Static word embeddings give one vector per word type
- This is useful, but it treats all uses of a word as if they had one dominant meaning
- See this example from Kroon *et al.* (2024)

Same word	Local context	Likely meaning
date	put a date in the lunchbox	fruit
date	went on a date	social event
date	what is the date today?	calendar time

- Contextual representations represent tokens **in context**, so the representation of date will differ across these examples.

From words to context

Core idea: a word vector should depend on the words around it.

N-grams

Short fixed windows

“not good”

“very good”

Useful, but also quite clunky.

Fixed size.

CNNs

Learn local patterns

A small filter scans over nearby words and learns which phrases matter.

Good at detecting expressions such as:

“not at all convincing”

RNNs

Read in sequence

The model reads word by word and carries a running memory forward.

Good for order and dependencies across a sentence.

Intuition: CNNs scan for meaningful local patterns; RNNs read text as an ordered sequence.

Transformers: context all at once

Transformers move beyond reading only left-to-right or scanning local windows, by using **attention**: each word can look at other words in the context and decide which ones matter most.

Example:

*The **bank** raised interest rates.*

*The boat reached the **bank** of the river.*

The same token, **bank**, receives a different representation because it attends to different surrounding words.

This way meanings produced from the whole sentence or passage.

Intuition:

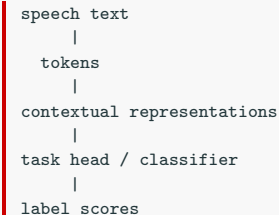
- CNN: “What useful phrase patterns appear nearby?”
- RNN: “What have I read so far?”
- Transformer: “Which words anywhere in the context matter for this word?”

Self-attention intuition

- Transformers rely on **self-attention**
- For each token, the model asks:
 - which other tokens in this text should I pay attention to?
 - how relevant are they for interpreting this token?
- Example:
 - in “She put a date in his lunchbox”
 - date should attend more to lunchbox than to calendar
- In essence: transformers create embeddings for each **token**, not just each word type, by focusing on relevant words in context.

Transformer models in a nutshell

- It breaks text into tokens
- It represents each token as numbers
- It updates those numbers using surrounding context
- It produces a representation that can be used for a task



```
graph TD; A[speech text] --> B[tokens]; B --> C[contextual representations]; C --> D[task head / classifier]; D --> E[label scores];
```

A vertical flowchart illustrating the Transformer model process. It starts with 'speech text' at the top, followed by 'tokens', 'contextual representations', 'task head / classifier', and finally 'label scores' at the bottom. Each step is connected to the next by a vertical line. A red vertical bar is positioned to the left of the flowchart.

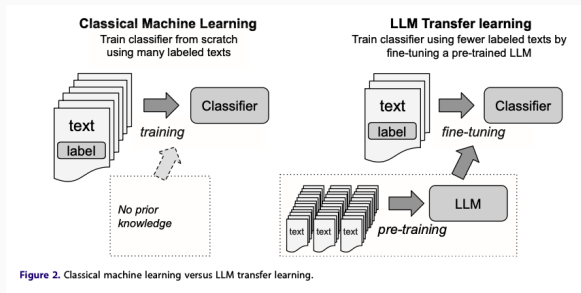
- The same pre-trained model can be adapted to many tasks

Pre-training transformer models

- A transformer is first trained on huge amounts of unlabelled text
- In BERT-style models, common pre-training tasks include:
 - **masked language modelling**: hide words and predict them from context
 - **next sentence prediction**: predict whether two text spans appeared sequentially
- This is where the model acquires general language representations

Transfer learning

- Transfer learning reuses knowledge learned from another task or corpus
- Now we can say what is being transferred:
 - learned language representations
 - sometimes task-specific abilities
- Typical workflow:
 1. select a pre-trained model
 2. fine-tune or adapt it to a task
 3. evaluate performance on held-out data



Transfer learning intuition

Stage	What the model learns	Why it helps us
Pre-training	general language patterns from large corpora	words, syntax, common associations
Fine-tuning	how to solve a task such as sentiment or topic classification	fewer local labels are needed

- Transfer learning does not remove measurement choices but it changes where they enter: model choice, label wording, corpus fit, and validation

What gets transferred?

Laurer *et al.* (2024) distinguish two kinds of useful prior information.

Type	Intuition	Example
Language knowledge	statistical patterns of language	NHS, hospital, and patient often occur in related contexts
Task knowledge	how to solve a general task	judging whether a context makes a hypothesis true, false, or neutral

- BERT-style models transfer **language knowledge**
- BERT-NLI also transfers **task knowledge** from natural language inference
- This is why NLI is attractive when local labelled data are scarce

Choosing a pretrained model

1. Define the prediction task
 - Does the model produce the output you need?
 - document classification, token classification, embeddings, or text generation
 - Do its labels match your concept?
 - A model trained for generic sentiment is not automatically valid for political support, stance, or evaluative tone.
2. Check the model's training context
 - Language: Which languages were included, and how well represented were they?
 - Domain: Was it trained or fine-tuned on news, social media, parliamentary speech, reviews, or general web text?
 - Unit of analysis: sentences, short posts, or long documents?
 - Label definition: How were categories defined and annotated?
3. Check practical compatibility
 - Model size, memory requirements, inference speed, and maximum input length

Using the Hugging Face Hub

- Hugging Face hosts model repositories containing model weights, tokenizers, configuration files, code, and often a model card. Models can be searched and filtered by task, language, library, licence, model size, and other metadata.

A model card may report:

- intended uses and limitations;
- architecture and training data;
- fine-tuning task and label meanings;
- evaluation datasets and performance;
- risks, biases, licence, and example code.

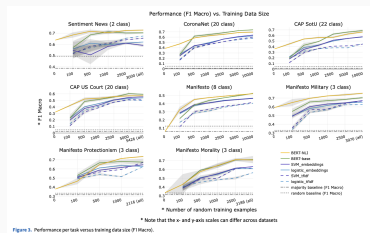
But availability on Hugging Face is not evidence of suitability. Researchers should still validate the selected model on data that reflect their own concept, language, domain, period, and class distribution.

BERT-NLI is one model choice

- **BERT** is not one single method for all tasks
- It is part of a family of transformer-based language models:
 - BERT, RoBERTa, DeBERTa, Longformer
 - domain-specific models such as legal, biomedical, or political models
 - multilingual models such as mBERT or XLM-R
- **BERT-NLI** is a BERT/DeBERTa-style model additionally trained on natural language inference
- Use BERT-NLI when categories can be expressed clearly as hypotheses and labelled data are scarce

BERT-NLI for political text analysis

- Laurer *et al.* (2024) compare transfer-learning approaches (BERT-NLI) with classical supervised machine learning
- Core argument:
 - NLI-style models can reduce annotation needs for many social science tasks



What Laurer et al. (2024) show

- The central problem is **data scarcity**
 - social science tasks are often custom, rare, and imbalanced
- BERT-NLI can work with fewer annotated examples because:
 - labels are verbalised as hypotheses
 - the model has seen many NLI-style text-hypothesis pairs
 - minority classes can benefit from prior language and task knowledge

Natural language inference

NLI models are trained to compare a **context** and a **hypothesis**.

Label / hypothesis	Context	NLI class
amnesty: "This speech says there was an amnesty."	"... as a result of the amnesty, we reckon that something in the region of 25,000 guns... are being handed in to the police."	entailment / true
amnesty: "This speech says there was an amnesty."	"... this scheme never offered an amnesty..."	contradiction / false
amnesty: "This speech says there was an amnesty."	"Will he protect per-pupil funding for schools in Nottingham?"	neutral

- The task is not to decide whether the hypothesis is objectively true
- The task is to decide whether the **context string** entails, contradicts, or is neutral toward the **hypothesis string**
- For classification, we choose the label whose hypothesis has the strongest entailment / true score

Zero-shot classification

- Zero-shot classification uses natural-language labels without local task-specific training examples
- The researcher supplies:
 - candidate labels
 - label descriptions or hypothesis templates
 - a validation design
- The model supplies scores for each label
- Good labels are clear, distinct, and close to the concept you want to measure

Sensitivity to prompts and labels

- Zero-shot predictions can change when we change:
 - label names
 - label descriptions
 - hypothesis templates
 - single-label vs multi-label settings
 - truncation or chunking choices

Takeaway

- Weak supervision and transfer learning change the economics of labelling
- The hard part of doing SML research shifts from training-data collection to:
 - concept definition
 - prompt and label design
 - baseline comparison
 - validation and error analysis
- Cheap labels are useful only when we understand their errors